

SE Lab-4 — LLM Chat History, Pramiti Ragavendra Udupa, PES1UG24CS331

Claude Code session, 1 Oct 2026

Session in which the completed Lab-4 code was explained task by task, tested headless, and summarised. Model reasoning is omitted (the export stores some brief progress notes between tool calls as reasoning, so a few of those are missing); tool actions are shown as one-line summaries.

Pramiti

[attached: Lab_4_VibeCoding_handout.pdf] [attached: VibeCodingGitRepoLinks-Sec-F.pdf] This is my SE Lab 4 (Bubble Bobble repair lab). The game is in Lab-4/game.py and the original buggy version is at github.com/SETAPESU26/26_bubbleBobble. Read both and give me a quick overview of what the lab asks for.

Claude (action)

Read — Downloads/Lab_4_VibeCoding_handout.pdf (pages 1-10) [failed]
Read — Downloads/VibeCodingGitRepoLinks-Sec-F.pdf (pages 1-10) [failed]
Glob — Lab-4/**
Bash — List repo, find game.py, check PDF libs [failed]
Bash — Extract text from both lab PDFs
Bash — Extract handout text with UTF-8 output
ToolSearch — select:WebFetch

Claude

I've read both PDFs. Next I'm pulling the original repo's README and game code from GitHub.

Claude (action)

Bash — Clone original repo to temp and show README
Bash — Show the bug site and the three stub functions

Claude

Lab 4 asks you to fix one bug in the Bubble Bobble game, add three features using an AI assistant, and submit before/after videos plus your chat log.

One thing first: Lab-4/game.py isn't in this working directory (C:\vdc\clone\bigdemo). This repo is a different project (it has app.py, api/, models/ and so on). Your Lab-4 work must be in another folder. I cloned the original repo into a temp folder to read it.

The handout (Lab 4: VibeCoding, 60 min, individual work)

1. Clone or fork your assigned repo. Yours is #26, SETAPESU26/26_bubbleBobble.
2. Run the game and record **10 seconds of video before** you change anything, showing the bug.
3. Use an AI assistant to fix the bug, then add each feature from the README one at a time. The handout says to aim for each job in **3–4 prompts or fewer**.
4. Record **10 seconds of video after** the changes.
5. Push to **your own repo** under a Lab-4/ folder. **Don't open a PR** to the SETAPESU26 repo.

Deliverables in Lab-4/: the before and after videos, the updated code, and your AI chat history exported as a doc or PDF. The README's checklist asks for a link to the chat page instead of an export, so check which one your instructor wants.

What the README asks for in game.py (271 lines, Pygame)

Controls: arrow keys to move, Up to jump, Space to blow a bubble, R to reset.

Task	What	Where
1. Fix the bug	Bubbles only trap an enemy on a near-perfect overlap. The code compares a <i>squared</i> distance to <code>24 * 2</code> (= 48). It should be <code>24 ** 2</code> (= 576).	<code>trap_enemies</code> , line 153
2. <code>bubble_tint(bubble)</code>	Return an (r, g, b) colour, or <code>None</code> for the default. Suggested: change colour as <code>bubble.life</code> runs out, and check <code>bubble.enemy</code> to tell trapped bubbles apart.	line 18
3. <code>on_fruit_collected(fruit)</code>	Called when the player picks up a fruit; the return value is ignored. Suggested: a sparkle effect or a fruit counter in the HUD.	line 23
4. <code>bonus_life_threshold()</code>	Return a score step that earns an extra life (e.g. 5000), or <code>None</code> to turn bonus lives off. The bookkeeping is already written.	line 28

The bug fix is a one-character change. Tasks 2 and 4 are a few lines each. Task 3 takes the most work if you add a HUD counter, because the function has no reference to the game object.

Since the lab grades how you prompt, I can explain each change or help you word your prompts without writing the code for you. Tell me where your `Lab-4` folder is if you want me to work on it.

Pramiti

Task 1: explain the bug in `trap_enemies`. Why does `24 * 2` make trapping almost impossible, and why is `24 ** 2` the right fix? Show me the exact line before and after.

Claude (action)

Bash – Find bubble and enemy sizes in original game
 Bash – Show bubble radius definition

Claude

The bug

The line is in `trap_enemies` (line 153 of the original `game.py`):

Before:

```
if bubble.pos.distance_squared_to(enemy.center) < 24 * 2:
```

After:

```
if bubble.pos.distance_squared_to(enemy.center) < 24 ** 2:
```

Why `24 * 2` makes trapping nearly impossible

`distance_squared_to()` returns the distance **squared** ($dx^2 + dy^2$). It skips the square root because that's slower, and comparing squared values gives the same answer. The catch is that both sides of the comparison must be squared.

- The author wanted to trap an enemy whose centre is within **24 pixels** of the bubble's centre.
- `24 * 2` is **48**, not 24 squared. So the code actually checks `distance2 < 48`, which means `distance <  $\sqrt{48}$` ≈ 6.9 px.

That makes the trap area about 3.5× narrower than intended and about 12× smaller in area (48 vs 576). For scale, an empty bubble has a radius of 14 px and an enemy is a 28×28 box, so they visibly touch when their centres are about 28 px apart. With the bug, the bubble's centre has to land within 7 px of the enemy's centre. Two other things make it worse:

- A bubble can only trap during its first 1.5 s (`bubble.age >= 1.5` returns early).
- The check only runs once per frame while the bubble moves fast forward, so it can jump straight over a 7 px window between two frames.

Why `24 ** 2` is right

`**` is Python's power operator, so `24 ** 2 = 576`. The check becomes `distance2 < 576`, which is exactly `distance < 24`. That's the 24 px radius the author meant: a bubble that clearly brushes an enemy traps it, and one that misses by a lot doesn't.

`< 24 ** 2` and `distance_to(...) < 24` give the same result. The squared form just avoids a square root on every bubble-enemy pair, every frame.

What to show in your video: before the fix, blow bubbles straight into an enemy and they pass through. After the fix, they trap it on contact.

Pramiti

Task 2: walk me through `bubble_tint` step by step. How are the colours chosen for empty bubbles vs bubbles with a trapped enemy, and how does the white flashing in the last 1.5 seconds work?

Claude (action)

Bash – Search for Lab-4 folder and `game.py` on disk
 Bash – Show `bubble_tint` implementation in Lab-4 `game.py`
 Bash – Show bubble life countdown and expiry logic

Claude

Your finished `bubble_tint` is at [Lab-4/game.py:25](#) (in `C:\Users\smith\el-clean`, not this repo). Here it is step by step.

Background: how a bubble's timers work

A bubble has two clocks ([game.py:130-138](#)):

- `bubble.age` counts **up** from 0 on every bubble, every frame.
- `bubble.life` starts at `BUBBLE_LIFE = 8.0` and only counts **down while an enemy is trapped** (`if self.enemy: self.life -= dt`). When it reaches 0, the enemy escapes ([game.py:227](#)).

So `life` means nothing for an empty bubble because it stays at 8.0. That's why the function uses a different clock for each kind of bubble.

Step 1: empty bubbles fade by age (lines 27-30)

```
fade = min(1.0, bubble.age / 3.0)
return (int(120 + 80 * fade), int(230 - 30 * fade), 255)
```

- `fade` goes from 0 to 1 over the first 3 seconds, then stays at 1.
- Red goes from 120 to 200, green from 230 to 200, and blue stays at 255.
- The bubble starts **bright cyan** (`120, 230, 255`), which is the game's default empty colour, and fades to **pale lavender-blue** (`200, 200, 255`). Older bubbles look "stale", and new ones still look like the original.
- It returns here, so empty bubbles never reach the flashing code.

Step 2: trapped bubbles use the fraction of life left (line 32)

```
ratio = max(0.0, min(1.0, bubble.life / BUBBLE_LIFE))
```

`ratio` is 1.0 just after trapping and 0.0 when the enemy is about to escape. The clamp keeps it in range even if `life` drops slightly below 0 in the frame it expires.

Step 3: a two-part colour fade (lines 33-38)

The 8 seconds are split in half. Each half fades between two colours:

ratio	Seconds left	Formula	Colour
1.0	8	first branch, $t=1 \rightarrow (255, 190, 230)$	pink (same as the default trapped colour)
0.5	4	first branch $t=0$ / second branch $t=1 \rightarrow (255, 230, 80)$	yellow
0.0	0	second branch, $t=0 \rightarrow (255, 60, 60)$	red

- **First branch (ratio > 0.5):** $t = (\text{ratio} - 0.5) / 0.5$ maps the top half of the range ($1.0 \rightarrow 0.5$) onto t from 1 to 0. Green rises from 190 to 230 and blue falls from 230 to 80, so pink becomes yellow.
- **Second branch (ratio ≤ 0.5):** $t = \text{ratio} / 0.5$ maps the bottom half onto t from 1 to 0. Green falls from 230 to 60 and blue from 80 to 60, so yellow becomes red.
- Both branches give exactly $(255, 230, 80)$ at 0.5, so the colour doesn't jump at the switch. Red stays at 255 the whole time, which keeps the colours warm, like a warning light.

Step 4: white flashing in the last 1.5 seconds (lines 40-41)

```
if bubble.life < 1.5 and int(bubble.life * 8) % 2 == 0:  
    return (255, 255, 255)
```

- `bubble.life < 1.5` turns the flashing on only in the final 1.5 s.
- `bubble.life * 8` cuts time into **1/8-second slices**, and `int(...)` gives the slice number: 11, 10, 9, ... 0.
- `% 2 == 0` is true on even slices. Even slices are **white** and odd slices show the **red/orange colour** from Step 3.

The result is a blink that changes every 0.125 s, which is **4 full on-off cycles per second**. That's 12 slices in total: 6 white and 6 coloured. Slice 11 (odd) comes first, so the flashing starts on a coloured frame. Slice 0 (even) comes last, so the bubble is white just before the enemy escapes.

The blink depends on `life` (game time), not on a frame counter, so it runs at the same speed at any frame rate.

How it connects to draw

The draw code ([game.py:266](#)) is `bubble_tint(bubble)` or `<default>`. Your function always returns a colour now, so the default after `or` is never used. That's fine, because each branch starts at the default colour anyway.

For your "after" video: trap an enemy and wait about 8 seconds. The bubble should go pink → yellow → red, blink white, and then the enemy escapes.

Pramiti

Task 3: explain `on_fruit_collected`. How are the sparkles created, how do they move with gravity and fade out, and where does the Fruits counter get reset when I press R?

Claude (action)

Bash – Find sparkle, counter, and reset code

Bash – Show reset, sparkle update, collect, draw, and key handling

Claude

Your `on_fruit_collected` is at [Lab-4/game.py:45](#). It works in four parts: create the sparkles, move them, draw them, and reset the counter on R.

Why the state lives in globals

The hook only receives the `fruit`. It gets no reference to the `Game` object, so it can't write `self.something`. Your version solves this with two **module-level** variables ([game.py:21-22](#)):

```
fruit_stats = {"collected": 0}
sparkles = [] # each sparkle: [x, y, vx, vy, life]
```

These are a dict and a list, which are **mutable**, so the function can change them in place (`+= 1`, `.append`) without a `global` statement. A plain `fruit_count = 0` would raise `UnboundLocalError` when you do `fruit_count += 1` inside the function.

1. Creating the sparkles (lines 45-50)

`update()` calls the hook right after adding the score and removing the fruit ([game.py:233-236](#)). Then:

```
fruit_stats["collected"] += 1
cx, cy = fruit.center
for _ in range(14):
    sparkles.append([cx, cy, random.uniform(-160, 160), random.uniform(-220, 40),
random.uniform(0.4, 0.8)])
```

- The counter goes up by one.
- **14 particles** are created, all starting at the fruit's centre.
- Each one is a 5-item list: `[x, y, vx, vy, life]`.
 - `vx` is random in $-160 \dots 160$ px/s, so sparkles spread left and right.
 - `vy` is random in $-220 \dots 40$ px/s. In Pygame, **y increases downward**, so a negative value means up. Most sparkles shoot **upward**, and a few start slightly downward. The result looks like a fountain.
 - `life` is random in $0.4 \dots 0.8$ s. Because each sparkle has a different lifetime, they disappear one at a time instead of all together.

2. Moving with gravity (lines 201-205)

At the start of every `Game.update`:

```
for spark in sparkles:
    spark[0] += spark[2] * dt    # x += vx*dt
    spark[1] += spark[3] * dt    # y += vy*dt
    spark[3] += 600 * dt         # vy += gravity*dt
    spark[4] -= dt               # life -= dt
```

This is basic **Euler integration**:

- Position changes by velocity $\times$ time.
- `vy` grows by **600 px/s every second**. That's gravity pulling down, because positive y is down. A sparkle launched at -220 slows down, stops at the top of its arc after about 0.37 s, and then falls. That's the curved arc you see.
- `vx` never changes (no air resistance), so the sideways spread stays constant.
- Everything is multiplied by `dt`, so the motion runs at the same speed at any frame rate.

The sparkle loop runs **before** the `if self.state != "play": return` check, so sparkles finish their animation even on the game-over screen.

3. Removing and "fading out" (lines 206, 273-275)

Dead sparkles are removed with:

```
sparkles[:] = [spark for spark in sparkles if spark[4] > 0]
```

The `[:]` replaces the list's contents **in place**. Writing `sparkles = [...]` instead would create a local variable inside `update` and fail the same way as the `UnboundLocalError` example above.

The fade isn't a transparency fade. The sparkle **shrinks** ([game.py:273-275](#)):

```
size = max(1, int(spark[4] * 5))
pygame.draw.circle(screen, (255, 240, 120), (int(spark[0]), int(spark[1])), size)
```

- The radius is `life × 5`. A sparkle with 0.8 s left is 4 px, one with 0.4 s left is 2 px, and near the end it's 1 px (the `max(1, ...)` floor). After that it's removed.
- The colour stays pale gold (255, 240, 120) the whole time. Shrinking from 4 px to 1 px reads as fading, and it's simpler than true transparency, which would need a separate surface with an alpha channel.

4. The Fruits counter and resetting it with R

Display: the HUD reads the counter every frame ([game.py:279](#)): `... Fruits {fruit_stats['collected']}`
....

Reset path:

1. The main loop catches the key ([game.py:302-303](#)):

```
elif event.type == pygame.KEYDOWN and event.key == pygame.K_r:
    game.reset()
```

1. `Game.reset()` ([game.py:156-163](#)) resets level, score, lives and combo, and you added two lines:

```
fruit_stats["collected"] = 0
sparkles.clear()
```

The first sets the counter to 0. The second removes any sparkles still in the air, so none are left over on the new game. `.clear()` changes the list in place, for the same reason as `[:]` above.

`reset()` is also called from `Game.__init__`, so the same code sets things up when the game first starts. Moving to the next level goes through `start_level()`, **not** `reset()`, so the counter carries over between levels and only R (or restarting the game) sets it to 0. That's probably what you want: it's a count of fruit for the whole run.

For your video: pop a trapped enemy, collect the fruit, and you'll see the gold burst and the HUD counter go up. Then press R and the counter goes back to 0.

Pramiti

Task 4: how does `bonus_life_threshold` give an extra life every 5000 points, and how does the "Next life @" text in the HUD work? Can you test it to prove a life is added at 5000?

Claude (action)

Bash – Show bonus life code and check pygame availability
Bash – Check constant value and update ordering

Claude

Your implementation reads simply, but I want to check the frame order and edge cases by actually running it. I'll write a headless test script in my scratchpad and leave your Lab-4 folder untouched.

Claude (action)

Write – `scratchpad/test_bonus_life.py`
Bash – Run headless bonus-life tests [failed]

Claude

That failure is in my test harness, not your game: pygame's `Font.render` can't be monkey-patched. I'll wrap the font object instead.

Claude (action)

Edit — scratchpad/test_bonus_life.py

Bash — Rerun headless bonus-life tests

Claude

Yes, it works: a life is added at exactly 5000 points. I ran your `game.py` without a window, set the score directly, and all 15 checks passed.

How the threshold works

Your function only returns a constant (`game.py:18` and `:53`):

```
BONUS_LIFE_SCORE = 5000
def bonus_life_threshold():
    return BONUS_LIFE_SCORE
```

The awarding code was already in the game. It runs every frame in `update()` (`game.py:213-216`):

```
threshold = bonus_life_threshold()
if threshold and self.score // threshold > self.bonus_awarded:
    self.bonus_awarded = self.score // threshold
    self.lives += 1
```

- `self.score // threshold` is integer division. It counts how many whole multiples of 5000 the score has reached: 4999 → 0, 5000 → 1, 9999 → 1, 10000 → 2.
- `self.bonus_awarded` stores how many multiples have been rewarded so far. It starts at 0 in `reset()`.
- A life is given only when the current count is **greater than** the rewarded count, and then the rewarded count is updated to match. That's what makes it "first time only": staying at 5000, or going back down and up again, doesn't give another life.
- `if threshold and ...` short-circuits. If the function returns `None` (or `0`), the division never runs, so bonus lives are off and there's no divide-by-zero.

How "Next life @" works

In `draw()` (`game.py:276-277`):

```
next_bonus = f" Next life @ {(self.bonus_awarded + 1) * threshold}" if threshold else ""
```

The next target is "the multiple after the last one rewarded" × 5000. It shows 5000 at first, 10000 after the first bonus, and so on. If the threshold is `None`, the string is empty and the HUD leaves it out. It's added to the HUD line next to Score, Lives and so on.

Test results

Check	Result
Score 4999	no life; HUD shows <code>Next life @ 5000</code>
Score 5000	lives 3 → 4 ; HUD shows <code>Next life @ 10000</code>
Stays at 5000 for 120 frames	still 4 (no repeats)
9999, then 10000	none at 9999, +1 at 10000
Real pickup: 200-point fruit at 4900	score becomes 5100; life added one frame later

R / reset()	bonus_awarded back to 0; HUD shows @ 5000 again
Threshold None	no life even at 50000; no "Next life" text

The "Level 2" in the HUD output above is a side effect of the test setup. I removed the enemies so nothing could hit the player, and the game counted that as a cleared level. It isn't a bug.

Two quirks worth knowing

1. **The life arrives one frame late.** The bonus check runs *before* fruit pickup within `update()`. So the frame where you collect the fruit adds the points, and the next frame (1/60 s later) adds the life. You can't see the delay.
2. **Skipping several thresholds in one frame gives only one life.** Going from 4900 to 15000 in a single frame gave +1, not +3, because `bonus_awarded` jumps straight to 3. Real fruit is worth `100 × combo`, so you'd need a huge combo for this to happen in play. If you wanted one life per threshold, it would be `self.lives += self.score // threshold - self.bonus_awarded`. This code wasn't part of your task, though, and the README only asks you to pick the threshold.

For your video: 5000 points takes a while to reach in normal play. To show it quickly, you could set `BONUS_LIFE_SCORE` to something like `300` while recording, then change it back to 5000 before you push.

The test script is in my scratchpad, outside your repo, so nothing in `Lab-4/` was changed.

Pramiti

Run the game headless for a few thousand frames with random inputs to make sure nothing crashes, and summarise all four changes in a few lines I could say in a viva.

Claude (action)

Bash – Show main loop and state transitions

Write – `scratchpad/fuzz_game.py`

Claude

I left a useless trap counter in that script. I'll fix it so it actually counts traps.

Claude (action)

Edit – `scratchpad/fuzz_game.py`

Bash – Run 60,000-frame headless fuzz of the game

Bash – Rerun fuzz with bonus threshold lowered to 200

Claude

Six runs reported a crash. Before blaming the game, I'll look at the tracebacks. My guess is that one of my own assertions doesn't allow for the one-frame delay I found earlier.

Claude (action)

Bash – Group crash tracebacks by error type

Bash – Locate the strict bonus assertion

Read – `scratchpad/fuzz_game.py`

Edit – `scratchpad/fuzz_game.py`

Edit – `scratchpad/fuzz_game.py`

Bash – Rerun fuzz at 5000 and 200 thresholds

Claude

Nothing crashed in 120,000 frames, and every bubble colour and score/lives check held. Your `Lab-4/` folder wasn't changed.

Fuzz test results

I ran your `game.py` with no window, calling `update()` and `draw()` every frame. Frame times were randomly 1/60, 1/30 or 0.05 s (0.05 s is the game's cap).

- **Two input styles:** random held keys with random jumps, and a simple bot that chases enemies, blows bubbles and jumps onto trapped bubbles. Each ran 5 seeds × 6,000 frames, with R pressed randomly and after game over.
- **Checked on every frame:**
 - `bubble_tint` always returns a valid 0–255 integer colour.
 - Lives and score never go negative.
 - Dead sparkles are always removed.
 - The player stays on screen.
 - The bonus-life count stays correct.
 - R resets the fruit counter, sparkles and bonus count.

Run	Frames	Traps	Pops	Fruit collected	Escapes	White-flash frames	Bonus lives	Result
Threshold 5000 (as submitted)	60,000	120	152	12	12	412	0	no crashes
Threshold 200	60,000	125	137	12	12	620	12	no crashes

At 5000, no run scored high enough to earn a bonus life (the best was 600). So I reran with the threshold at 200 to make sure the bonus-life code actually ran during play.

The test-script failure along the way: that 200-point run first reported 6 "crashes". They were all my own assertion failing, not the game. My check expected the life on the same frame as the fruit pickup. It actually arrives one frame later, which is the delay I mentioned in Task 4. I changed the check to allow exactly one frame. All 12 bonus lives then arrived exactly one frame after the threshold was crossed, never later.

Viva summary

1. **Bug fix:** `trap_enemies` compared a squared distance to `24 * 2`, which is 48. That made the trap radius about 7 px instead of 24 px, so bubbles almost never trapped. I changed it to `24 ** 2` (576) so both sides of the comparison are squared.
2. **`bubble_tint`:** empty bubbles fade from cyan to pale blue over 3 seconds, based on their age. Trapped bubbles go from pink to yellow to red as their life runs out. In the last 1.5 seconds they blink white, switching every eighth of a second, to warn that the enemy is about to escape.
3. **`on_fruit_collected`:** it adds one to a fruit counter shown in the HUD and makes a burst of 14 sparkles at random speeds. Gravity pulls them into arcs, and they shrink until they disappear. The counter and sparkles are module-level variables because the hook gets no game object, and `reset()` clears both when you press R.
4. **`bonus_life_threshold`:** it returns 5000. The game compares `score // 5000` with the number of bonus lives already given, so each multiple of 5000 gives exactly one life. The HUD shows the next target as `(bonus_awarded + 1) * 5000`.

Both test scripts (`fuzz_game.py` and `test_bonus_life.py`) are in my scratchpad folder, outside your repo.

Pramiti

Export this session's transcript as a readable PDF named `Chat_History.pdf` (my messages labelled "Pramiti", heading "SE Lab-4 — LLM Chat History, Pramiti Ragavendra Udupa, PES1UG24CS331"), replace `Lab-4/Chat_History.pdf` with it, and push it to main of my EXPERIENTIAL-LEARNING repo as a normal commit.

```
Bash – Check repo remote, branch, status, and PDF libraries
Bash – Inspect existing Chat_History.pdf before replacing it
Bash – Read rest of existing chat PDF, Lab-4 README and diff
ToolSearch – select:mcp__ccd_session_mgmt__export_transcript
```

Transcript exported at this point; the PDF was then added to Lab-4 and pushed.